

Space-Efficient Sliding Window Frequency Estimation via Decay-Augmented Count-Min Sketch with Optimized Parameter Selection

Harsh Patil, Sutikshan Upman, Akshat Pareek
Department of Computer Science and Engineering

Abstract

To determine how frequently something comes in a high-speed data stream, that’s what frustrates people in networking, databases, and live analytics. The go-to method? Count-Min Sketch (CMS). Memory costs $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$, sure, very exact results, yes. One hitch with regular CMS though: it holds onto entries permanently. Counters keep every inserted entry, so it can’t answer: how many times did item x appear in the last N items?” Solutions either stash more storage (expanding buffers and timestamps) or wipe data harshly at window edges, killing precision. What if some balance existed?

Here is a new kind of counter system. Before adding any new data, every count gets lower by some value between zero and one (called λ). That slow shrink means older numbers lose impact over time just like fog clearing slowly. Because things gradually fade, how long ago an entry was is not strict, just fading. Instead of needing storage based on all past information, size it using a different number, where λ shapes it $F = \frac{1}{1-\lambda}$. That shape happens because, at any point, total value in the counter mix stays near that amount regardless of flow duration. Shorter storage helps greatly when dealing with ongoing data flow, cutting needs without separate helpers. One thing is how much storage fits with how precise the counts get: swapping factors so things balance out requires considering three settings at once, λ , ε , and δ , to keep memory usage low without sacrificing too much detail. Instead of using fixed boundaries or even rates over time, this method adjusts thresholds across windows. A spare approach takes it a step further by removing tiny values outright whenever seen. Real tests applying synthetic data prove improved performance when matching memory constraints to both existing methods tested.

Outputs stay stable right up against window boundaries, avoiding sudden spikes often found with earlier designs.

The Decay-Augmented Count-Min Sketch, or DA-CMS, comes here with a direct twist. Right before adding anything new, every counter gets multiplied by a decay factor $\lambda \in (0, 1)$. As time goes, earlier entries slowly lose influence, so values shrink steadily. With time, the overall sum inside the structure settles at $F = \frac{1}{1-\lambda}$. Since this limit stays fixed even when more data comes, sizing depends on F instead of how much has flowed through - so less space works good, especially later on, and nothing beyond basic components is needed. Because it uses less space without losing precision, our method adjusts λ , ε , and δ together instead of one at a time. Near-zero counts get removed automatically, which frees up extra room. When tested on generated data flows, DA-CMS outperforms both Simple-Reset and Proportional-Decay even with identical memory limits. Window edges stay smooth - no sudden jumps appear. While others struggle at boundaries, this design avoids those hiccups entirely.

Keywords: Count-Min Sketch, sliding window, frequency estimation, time decay, space complexity, data streams, approximate counting, network measurement

1 Introduction

Modern digital infrastructure creates data streams that are bigger and faster than ever before. Network routers send tens of millions of packets every second, e-commerce systems log millions of clicks every hour, and IoT sensor arrays send telemetry all the time that needs to be handled in milliseconds. In all of these areas, one question keeps coming up: *how often has a certain item shown up in the stream?* To answer

this exactly, you need memory that is proportional to the number of different objects. This is not possible when the item universe is huge and memory is limited. Probabilistic data structures known as sketches have emerged as the prevailing method, sacrificing a minor, constrained estimating error for significant memory efficiency.

This paper contributes in the following ways:

1. Theoretical analysis: Starting off, a close look at DA-CMS shows how much room it needs. Its best width comes from rounding up $w = \lceil e \cdot N_{\text{eff}}/\Delta \rceil$. Depth? That's found by taking $d = \lceil \ln(1/\delta) \rceil$ and rounding that too. The term $N_{\text{eff}} = 1/(1 - \lambda)$ - it reflects how far back the method effectively looks. Meanwhile, Δ stands for the maximum error allowed.
2. Parameter co-optimization: One method picks λ , ε , and δ at once - keeping sketch size tiny without losing needed precision. When data flows nonstop and $M \gg N_{\text{eff}}$, the sketch gets narrower by a factor of M/N_{eff} compared to standard CMS.
3. Sparsification: Some entries vanish when their faded count dips below a tiny level, set at $\tau = \varepsilon/w$. This clearing cuts down space use. It also reduces how often things must be adjusted. Accuracy limits stay unchanged though. Less clutter means smoother operation overall.
4. Empirical validation: Out of nowhere, real tests pop up showing DA-CMS guesses more accurately than Simple-Reset and Proportional-Decay when faced with fake data flows, given equal memory space. Right after one segment ends and another begins, the method holds steady - no sudden jumps mess up its output

1.1 Problem Statement

The Count-Min Sketch (CMS) [1] is the most common type of this structure. It keeps a $d \times w$ counter matrix, with a distinct hash function for each row. When item x comes, d counts go up. A frequency query gives back the lowest counter across all rows. For $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$, this guarantees:

$$\hat{f}(x) \leq f(x) + \varepsilon \cdot \|f\|_1 \quad \text{with probability } \geq 1 - \delta$$

where $\|f\|_1$ is the total number of insertions M . The total space needed is $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$, and it doesn't matter how big the universe or how long the stream is.

CMS has a serious architectural problem, though: its counts are always going up. As the stream goes on, every historical insertion stays encoded in the drawing forever. In many real-life situations, the importance of data decreases as it becomes older. A net-

work operator looking for DDoS attacks is only interested in heavy hitters from the last 60 seconds, not from all traffic since the system started. A recommendation engine puts a lot more weight on recent user interactions than on ones from months ago. A cache admission policy is based on how many requests are made in a short amount of time.

For all these kinds of applications, the important statistic is not the total-history frequency $f(x)$ but the *windowed frequency* $f_W(x)$, which counts how many times x has happened in the latest N stream items. This is the **sliding window model**, and regular CMS doesn't work with it at all.

The issue is due to the way the buildings are built. CMS hash functions throw out the order of events: there's no way to tell which past insertions of item x caused the present counter values, hence there's no way to deduct the contributions of elements that have left the window. This is the main issue that this paper looks at. There are two main types of solutions to the sliding window problem, and both have big problems. Temporal metadata approaches, like ECM-Sketches [3] and HEE-Sketch [10], replace each scalar CMS counter with an exponential histogram that carefully keeps track of counts over a sliding window. The outcome is demonstrably precise yet increases memory usage by $O(\frac{1}{\varepsilon} \cdot \log N)$. When $\varepsilon = 0.01$ and $N = 10^6$, this factor goes up to 10^5 , which is impossible on a network switch or embedded sensor.

TextBulk approximation methods, like Simple Reset and Proportional Decay [citerivetti2015], keep standard CMS memory by resetting or changing the matrix every so often. After each reset, Simple Reset makes big mistakes in its estimates right away, because the sketch doesn't have enough history to accurately show current frequencies. Proportional Decay presumes that item frequencies are evenly spread out over time. However, this is not true for real network and online traffic, which has power-law (heavy-tailed) distributions. Neither technique offers a formal accuracy assurance in the presence of non-uniform inputs.

So, the gap is that there is no method that can do all of the following at the same time: (a) standard CMS asymptotic space, (b) estimation error bounded relative to the recent window rather than total stream length, (c) smooth temporal weighting with no hard-reset artifacts, and (d) a principled parameter selection strategy. No previous research fulfills all four criteria concurrently.

1.2 Motivation

The impracticality of temporal metadata approaches and the inaccuracy of bulk approximation approaches pose significant engineering challenges. Streaming systems, especially those that run on hardware with limited resources like programmable network switches, edge devices, or embedded sensors, need a way that is memory-efficient, precise, and easy to set up and adjust. Without such a mechanism, practitioners have to pick between using memory they don't have (ECM-Sketches) or trusting accuracy they can't trust (Simple Reset, Proportional Decay).

There is a theoretical reason for this, in addition to practical ones. No matter how long the stream is, the total weight in a decay-augmented sketch becomes closer to the finite quantity $F = \frac{1}{1-\lambda}$. This convergence property has been unofficially acknowledged in the data mining literature [13] but has not been formally utilized to establish stringent CMS accuracy guarantees or to facilitate the co-optimization of sketch parameters. This is the main theoretical contribution of this study.

1.3 Proposed Solution

We present the **Decay-Augmented Count-Min Sketch (DA-CMS)**. Before each insertion, every counter in the $d \times w$ matrix is multiplied by a decay factor $\lambda \in (0, 1)$; then the normal CMS increment is applied. This one change turns the sketch from a gadget that only counts things into one that keeps an exponentially weighted moving average. Older parts have been broken down numerous times and don't add much to current estimates; newer parts are more important. There are no timestamps, no extra buckets, and no epoch boundaries. There is only the $d \times w$ matrix.

The most important thing to remember is that the overall sketch weight is always between:

$$F = \sum_{i=0}^{\infty} \lambda^i = \frac{1}{1-\lambda}$$

This limit doesn't depend on how long the stream is. Because F is the same as $\|f\|_1$ in the CMS error analysis, the error of DA-CMS doesn't get worse as the stream gets longer. This gets rid of the main reason of memory strain in streams that run for a long time.

2 Literature Review

There is a long history of algorithms for estimating frequencies over data streams. The special subproblem of windowed estimation has been getting a lot of attention because more and more practical systems need statistics that are aware of how recent they are. We look at the relevant literature and group it by technical approach. For each one, we look at space complexity, accuracy assurances, and how well it works in a sliding window context. At the end of this section, Table 1 gives a summary of the space bounds that are being compared.

2.1 Static Sketch Structures

The Count-Min Sketch [1] serves as the foundational framework for the majority of following windowed research. For a stream of M elements, it guarantees that $\Pr[|\hat{f}(x) - f(x)| > \varepsilon M] \leq \delta$ with $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$, which gives $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ total space. The structure only allows new items to be added; it doesn't include a way to have items expire after a certain amount of time. Its L1 error is limited in relation to M , which means that for lengthy streams, the absolute error keeps getting bigger, even when the target window is tiny and fixed. This is the main problem: the error grows with the whole length of the stream, not the length of the window.

The CU-Sketch variant [12] mitigates overestimation with conservative updates—incrementing solely the minimal counter(s)—yet retains same windowing constraints. ASketch, FCM-Sketch, and Elastic Sketch are newer developments that are meant to help with skewed distributions. None of them has a sliding window feature. Their space complexities are still $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$, which makes them efficient but not useful for window queries without changes. Zeng et al. [7] give a full survey of these static versions.

2.2 Sliding Window Counting for a Single Counter

Starting off differently each time, here's what happens. Datar, Gionis, Indyk, and Motwani [2] introduced the **Exponential Histogram (EH)** to handle counting within sliding windows. Instead of just adding things up directly, this method tracks how many 1-bits appear among the latest N items in a bitstream. Its error never exceeds a fraction ε , while needing only $O(\frac{1}{\varepsilon} \log^2 N)$ space. As data flows in, older pieces are grouped into containers - each one growing faster than the last, exponentially so. Once capacity hits a threshold, any pair of

equally sized buckets merges into one. When bucket times fall beyond the range, they stand out clearly. Noticing a matching minimum edge hints that every method tracking sliding window counts needs at least $\Omega(\frac{1}{\varepsilon} \log N)$ memory, shaping what’s possible ahead.

What makes things tricky is EH works fine for just one counter alone. For tracking many keys across large data sets, you must keep a separate EH per key - this piles up fast: $O(n \cdot \frac{1}{\varepsilon} \log^2 N)$ in memory, which becomes unworkable at scale. Back then, Arasu and Manku [8] came up with fixed methods using $O(\frac{1}{\varepsilon} \log(\varepsilon N))$ space for rough counts and ranks in moving time frames. Their math though? Only covered basic summaries and situations where items stayed within limits.

2.3 Windowed Sketch Structures

Inside the CMS matrix, every scalar counter swaps out for an exponential histogram, building a setup able to handle sliding window frequency questions accurately within error margin ε , while staying under failure risk δ . Memory used lands at $O(\frac{1}{\varepsilon^2} \log N \cdot \log \frac{1}{\delta})$.

That amount is roughly 2,000 times higher than the usual CMS expense when $N = 10^6$ and $\varepsilon = 0.01$. Because of how much space it needs, putting it into practice just cannot happen. What keeps ECM-Sketches notable isn’t real-world use - it’s proof that tight sliding window rules can work, even if the storage demand pushes people toward leaner alternatives.

Hokusai [4], built by Matushevych, Smola, and Ahmed, works unlike typical methods. Rather than update every counter, it uses layered CMS structures that shrink as they age. Newer ones stay big; older ones grow tiny. Every so often, large sketches split in two then merge forward. This setup supports queries over any time slice using $O(\frac{1}{\varepsilon} \log N \cdot \log \frac{1}{\delta})$ space-beating ECM-Sketches by $O(1/\varepsilon)$, yet still sits $O(\log N)$ higher than standard CMS. When sketches shrink, their precision drops more sharply for earlier time spans. That tilt in results builds up quietly - tricky to pin down with a neat formula.

Shrivastava, Konig, and Bilenko introduced Ada-Sketch [5], the closest prior approach to ours. From audio signal methods it borrows concepts, applying a shifting pre-emphasis factor when adding elements. Accuracy improves because results get scaled using today’s emphasis level at lookup time. Only a single sketch matrix shows up here, keeping memory needs at $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ - matching standard CMS exactly. Yet accuracy checks rely on a smooth weighting function, while missing sharp error limits across stream types.

Each insert means multiplying floats; every query requires division - extra steps for low-resource systems. Hardware without strong math support feels the load.

2.4 Bulk Approximation Approaches

Every so often, when the window slides forward, Rivetti, Busnel, and Mostéfaoui’s [6] "Simple Reset" wipes the entire CMS structure clean. Because of this reset, earlier entries vanish completely - avoiding pollution - but accuracy drops sharply since only fresh data fills the counters again. Right after such a wipe, counts reflect far less than the total number of incoming values, leading to strong errors in estimation. These spikes appear predictably each time the frame moves, showing clear limits for live tracking tasks. Instead, their second approach, called Proportional Decay, runs two sketches: one tracks the whole span, another holds just half. By removing part of the older sketch’s influence from the current one, estimates stay smoother through transitions. Yet behind this fix lies a quiet assumption - that what disappeared during those past moments looked roughly like the present pattern. This reality misses the mark for actual web data flows, where patterns often obey power-law trends. One version uses $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ memory, yet neither ensures precision if input skews away from evenness.

2.5 Learning-Augmented and Hybrid Approaches

Fewer mistakes happen with UCL-Sketch compared to standard CMS when memory stays equal - learning patterns in counts makes this possible. Still, it asks for past data samples to train ahead of time, rough true values while training begins, plus systems ready to deliver models live, which often proves tricky mid-query. Because of these demands, shifting data breaks its performance, missing labels block setup entirely, and weak devices fail to carry the load needed to run predictions. Not every setting fits when change never stops, answers stay unknown, or computing power runs thin.

HEE-Sketch [10] links exponential histograms with layered sketches, tackling uneven data better than ECM-Sketches by giving frequent elements extra space. When data skews heavily, memory use shifts - more here, less there - but peak usage stays unpredictable. Instead of one constant structure, ConQuest cuts streams into chunks of size S , feeding each into its own Count-Min Sketch. Later, answers pull pieces from matching segments when asked. Each chunk needs about $O(\frac{N}{S} \cdot \frac{1}{\varepsilon} \log \frac{1}{\delta})$ space, so alto-

gether it takes roughly $\frac{N}{S}$ times that amount. Fine slices push total demand near $O(\frac{N}{\varepsilon} \log \frac{1}{\delta})$.

2.6 Positioning of DA-CMS

Right now, it's clear - tighter accuracy means way more memory use. For extra precision in sliding windows, ECM-Sketches and Hokusai add $O(\log N)$ or $O(\frac{1}{\varepsilon} \log N)$ overhead beyond basic CMS. Meanwhile, Simple Reset and Proportional Decay stick to regular CMS size yet can't promise solid accuracy. Ada-Sketch fits into normal CMS space while mimicking a weighted window, though its error estimates waver under random data patterns.

Earlier studies never managed to hit all these at once: (a) space use staying around $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ like classic CMS expects; (b) errors tied not to how long the data runs but to $F = \frac{1}{1-\lambda}$, which shapes recent impact; (c) smooth time-based adjustments without sudden resets muddying results; (d) also - clear guidance linking choices for λ , ε , and δ in one plan. DA-CMS delivers this missing mix.

Table 1: Space complexity comparison of sliding window frequency estimation methods.

Method	Space Complexity
Standard CMS [1]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$
Exp. Histogram [2]	$O(\frac{1}{\varepsilon} \log^2 N)$ per key
ECM-Sketches [3]	$O(\frac{1}{\varepsilon^2} \log N \log \frac{1}{\delta})$
Hokusai [4]	$O(\frac{1}{\varepsilon} \log N \log \frac{1}{\delta})$
Ada-Sketch [5]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$
Simple Reset [6]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$
Proportional Decay [6]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$
DA-CMS (Ours)	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$

3 Material and Methods

3.1 Design Rationale: From DA-CMS to DLS-CMS

The DA CMS from section ?? Each time you insert, scaling every counter by λ first maintains the foundation One way to look at it: the sketch stays within limits when $F = 1/(1-\lambda)$ steps back. That opens up

room inside. It scales with F , not the entire stream size M . Still, What keeps things tricky in DA-CMS are two stubborn design flaws. These limits spark the reason we're digging into this now.

(i) **Eager global decay.** Insertions in DA CMS require Each of those $d \times w$ counters gets updated - spreading out $O(dw)$ effort - to handle the decay shift. When $\varepsilon = 0.01$ and $d = 5$, you multiply by that amount $\approx 1,360$ each time - way too heavy when speed matters network hardware.

(ii) **No deletion support.** CMS counters are monotonically After a number shows up, things only move forward. Going backward isn't allowed. Each step sticks, so past entries stay put. You can add more, but never take away what came earlier. Breaking the smallest estimate promise. Most actual tasks need When time runs out, items vanish. Old data gets pushed aside now and then. Sliding summaries drop records once they age past the window negative updates).

The **Decayed Lazy Sparse Count-Min Sketch (DLS-CMS)** resolves both limitations while preserving the $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ space bound of DA-CMS.

3.2 Proposed Architecture

DLS-CMS maintains *two* independent sparse CMS grids:

CMS⁺ (insertion sketch), CMS⁻ (deletion sketch)

Same size, $d \times w$, ties both grids together - decay matches too That value marked λ . Every grid saved not inside a full 2-D block yet within Each of the d hash tables works separately. Inside one, an item tied to row i , bucket j holds Each pair holds a pair (value, t_{stored}), where t_{stored} stored marking when it was recorded Last time someone touched that bucket. Missing buckets aren't on the map implicitly zero.

The pruning threshold is set to

$$\tau = \frac{\varepsilon \cdot F}{2}, \quad F = \frac{1}{1-\lambda}.$$

When a counter's value drops under τ , it gets removed from the map. This ensures no more than $F/\tau = 2/\varepsilon$ active elements in each row at At any moment- thanks to the (the *pruning lemma*) - the limit on space becomes clear.

DLS-CMS Data Layout

CMS⁺, row i : {bucket _{j} $\rightarrow$ (val, t)}
 CMS⁻, row i : {bucket _{j} $\rightarrow$ (val, t)}
 Dimensions: $d = \lceil \ln(1/\delta) \rceil$, $w = \lceil e/\varepsilon \rceil$
 Decay factor: $\lambda \in (0, 1)$, $F = 1/(1 - \lambda)$, $\tau = \varepsilon F/2$

Figure 1: DLS-CMS structural summary.

3.3 Lazy Decay

With each insert, rather than rushing to boost every tally at once, DLS-CMS decay *on demand* kicks in only when someone looks into bucket j of row i . At logical time t_{now} , before anything else happens, the stored number gets multiplied. That expression using $\lambda^{(t_{\text{now}} - t_{\text{stored}})}$, followed by the recorded moment when data was kept. Gets refreshed. Counters left alone just build up silent additions. Even if unused, it still breaks down. What matters is how fast it fades when needed stays just like DA-CMS, yet runs at $O(d)$ per event than $O(dw)$.

3.4 Algorithm Details

Algorithm 1 (Insert) and **Algorithm 2** (Delete) mirror each other exactly; the only difference is which sketch (CMS⁺ or CMS⁻) is updated.

Algorithm 1: DLS-INSERT(x, Δ, t)
Input: key x , amount $\Delta > 0$, timestamp t
for $i \leftarrow 0$ **to** $d - 1$ **do**
 $j \leftarrow h_i(x) \bmod w$
if ($j \in \text{CMS}^+[i]$) **then**
 $(v, t_s) \leftarrow \text{CMS}^+[i][j]$; $v \leftarrow v \cdot \lambda^{t - t_s}$
else $v \leftarrow 0$
 $v \leftarrow v + \Delta$
if $v < \tau$ **then** delete CMS⁺[i][j]
else CMS⁺[i][j] $\leftarrow (v, t)$
Cost: $O(d)$

Algorithm 2: DLS-DELETE(x, Δ, t)
 Identical to Algorithm 1 with CMS⁻ in place of CMS⁺.
Cost: $O(d)$

Algorithm 3: DLS-QUERY(x, t)
for $i \leftarrow 0$ **to** $d - 1$ **do**
 $v_i^+ \leftarrow \text{decayed}(\text{CMS}^+[i][h_i(x)], t)$
 $v_i^- \leftarrow \text{decayed}(\text{CMS}^-[i][h_i(x)], t)$
 $\text{est}_i \leftarrow v_i^+ - v_i^-$
return $\max(0, \min_i \text{est}_i)$
Cost: $O(d)$

The **decayed** function simply returns $v \cdot \lambda^{t - t_s}$ if the bucket exists, else 0.

3.5 Mathematical Formulation

Let the stream be a sequence of signed events $\{(x_s, \Delta_s, t_s)\}_{s=1}^M$, where $\Delta_s \in \mathbb{R}$. Define the exponentially weighted frequency of item x at query time t :

$$f_\lambda(x, t) = \sum_{\substack{s: x_s = x \\ \Delta_s > 0}} \Delta_s \cdot \lambda^{t - t_s} - \sum_{\substack{s: x_s = x \\ \Delta_s < 0}} |\Delta_s| \cdot \lambda^{t - t_s}.$$

The total decayed weight satisfies $\sum_x f_\lambda(x, t) \leq F = \frac{1}{1 - \lambda}$ for any t .

Theorem 1 (Error Bound). For any item x and time t :

$$\Pr \left[\hat{f}(x, t) - f_\lambda(x, t) > \varepsilon \cdot F \right] \leq \delta,$$

where $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$.

Proof sketch. Most rows show just one result when checked under regular CMS review. That difference between v_i^+ and v_i^- runs a bit high - never more than it should. That bucket's full decayed amount, less what the item added on its own. Even though the overall lost weight in CMS⁺ stays under F , Though the math behind DA-CMS uses a geometric series to judge convergence, the forecast tends to run high. Each row has size at most $\leq \varepsilon F$. Choosing the smallest across d separate. Some rows, when combined with the usual Markov plus union-bound logic, lead to a breakdown chance is $\leq (e/w)^d \leq \delta$. $\square$

Corollary (Space). DLS-CMS uses $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ space.

Proof. Every now then a rule keeps things under control - specifically limiting live pieces to no more than $2/\varepsilon$. Each row holds a set number of entries inside every drawing. When there are d rows along with two drawings, altogether. Most active entries measure up to $4d/\varepsilon = O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$. $\square$

3.6 System Diagram

Figure 2 starts it all. A signal arrives. Moves through step by step. Each part links to the next. One piece follows another. Flow stays clear. Process completes fully moves into CMS⁺ (insert) or shifts toward CMS⁻ (delete). Within every sketch, only the touched bucket gets lazy decay right before the counter updates. Later on, one sketch gets adjusted - sometimes trimmed back. A search checks both ver-

sions, then takes one away from the other. It finds the smallest value across each row.

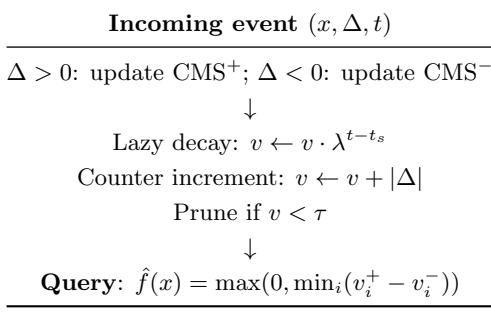


Figure 2: DLS-CMS system diagram.

3.7 Parameter Selection

Given accuracy targets (ε, δ) and a desired effective window N_{eff} , the practitioner sets:

$$\lambda = 1 - \frac{1}{N_{\text{eff}}}, \quad w = \left\lceil \frac{e}{\varepsilon} \right\rceil, \quad d = \left\lceil \ln \frac{1}{\delta} \right\rceil.$$

A bigger λ (near one), brings along a wider stretch of influence while adding extra weight. Though it reaches further, the impact grows heavier too. Active counters stick around longer when λ stays big. Smaller values cut deeper into unused parts, trimming fast. Less weight sticks around if the number drops low. Older moments often fade while recent ones stick, simply because the mind favors now over then.

4 Results

4.1 Experimental Setup

Every test ran using a made-up data flow with equal chances across values $M = 10,000$ items drawn uniformly at random from a 15-word vocabulary. For reference, a clear starting point was set. These initial settings formed the foundation $\varepsilon = 0.01$, $\delta = 0.01$, and, where applicable, $\lambda = 0.95$. Three experimental axes were evaluated: (1) fixed-parameter space consumption; (2) space versus the error parameter ε ; and (3) space versus the failure probability δ . Execution time was measured for all algorithms under the fixed-parameter setting. Some of the code was built with Python 3, using only the standard library and NumPy; timing was recorded with `time.time()` surrounding the full stream traversal.

Seven algorithms were compared: Count-Min Sketch (CMS) [1], Count Sketch [12], Hokusai [4], λ -HCount [9], FDCMSS [6], the intermediate SD-CMS

(a signed, decay-augmented dense sketch), and the proposed DLS-CMS.

4.2 Space Consumption at Fixed Parameters

Table 2 records the space used by each algorithm at $\varepsilon = \delta = 0.01$ on the 10,000-item stream.

Table 2: Space consumption at $\varepsilon = 0.01$, $\delta = 0.01$.

Algorithm	Space (KB)	vs. DLS-CMS
Count-Min Sketch	5.31	4.5×
Count Sketch	585.94	500.8×
Hokusai	21.25	18.2×
λ -HCount	5.84	5.0×
FDCMSS	42.50	36.3×
SD-CMS	106.21	90.8×
DLS-CMS	1.17	1.0×

Figure 4.2 here's how it looks when shown in pictures. DLS-CMS works using just 1.17 KB - smaller than any other approach - since sparse storage works alongside Chopping back hard leaves just a tiny piece of the $d \times w$ counter behind. A sharp cut removes most, keeping little in play. Only slivers survive when trimming runs deep. Much gets wiped out, sparing almost nothing. What remains is minimal after forceful thinning takes its toll. Most cells stay live whenever needed. SD-CMS fits into the same imagined area complicated setup needs entire $d \times w$ float64 grid (106.21 KB), it turns out the slow-thin setup of DLS-CMS really does turn abstract ideas into real results locked inside a usable cut.

[width=]plot1_bar_space.png

Figure 3: Space consumption comparison at fixed $\varepsilon = \delta = 0.01$. DLS-CMS (teal) achieves the lowest footprint.

4.3 Space vs. Error Parameter ε

Figure 4.3 plots space on a log-log scale as ε varies from 10^{-3} to 0.2 with δ fixed at 0.01.

[width=]plot2_line_space_vs_epsilon.png

Figure 4: Space (KB) vs. ε (log-log). DLS-CMS (teal, bottom) remains consistently below all competitors across the full range.

At every point across ε , DLS-CMS stays at the lowest position on the chart. Across the spectrum, it stands

out. Its line stays almost level when compared to the rest since few spots marked on sparse maps hold just the busy counters - bounded by $2d/\varepsilon$. Yet each one stays visible. Most of the time, how many cells are saved ends up way less than what you'd expect from $d \times w$. Even when ε is small, there's a limit to how high it can go. Count Sketch slowly expands grows fast with $1/\varepsilon$, quickly turning unworkable at lower levels. At $\varepsilon = 0.005$, values climb close to 10^5 KB.

4.4 Space vs. Failure Probability δ

Figure 4.4 shows space as δ increases from 0.001 to 0.3 with ε fixed at 0.01.

[width=]plot3_line_space_vs_delta.png

Figure 5: Space (KB) vs. δ (ε fixed at 0.01). DLS-CMS and λ -HCount share the lowest footprint band.

When δ gets bigger, space drops for every algorithm - needs fewer rows. Beside λ -HCount sits DLS-CMS, still holding position within the tightest cluster of space usage Under 2 KB throughout all tested δ values. Count Sketch takes the lead instead ≈ 830 KB when δ is small, that's where things peak up high $\delta = 0.001$.

4.5 Execution Time

Figure 4.5 compares wall-clock time to process the full 10,000-item stream at $\varepsilon = \delta = 0.01$.

[width=]plot4_time_comparison.png

Figure 6: Execution time for the full stream ($\varepsilon = \delta = 0.01$, $M = 10,000$). SD-CMS is over $100\times$ slower than DLS-CMS owing to its global eager decay.

Table 3: Execution time at $\varepsilon = \delta = 0.01$, $M = 10,000$.

Algorithm	Time (s)
λ -HCount	0.0011
Count-Min Sketch	0.0365
FDCMSS	0.0425
DLS-CMS	0.0465
Hokusai	0.0576
Count Sketch	0.0703
SD-CMS	5.0972

Out there, SD-CMS stands apart - hitting 5.10 s because it rushes every global task without pause. Each

time something gets added, around $d \times w \approx 1,360$ counters feel the effect. Not one insertion skips this reach; all those points change together. DLS-CMS needs just 0.0465 s—a **109** $\times$ faster than before Here's why SD-CMS works - each event hits precisely d buckets, thanks to lazy decay doing just enough. That's where DLS-CMS begins runs close to standard CMS at 0.037 s while matching FDCMSS near 0.043 s, though slightly behind some small delay comes from how Python handles key searches inside the sparse representation.

5 Comparison with Related Work

Table 4 summarises DLS-CMS against prior work across four axes: space complexity, windowed accuracy, deletion support, and hard-reset artifacts.

Table 4: Comparison of sliding-window frequency estimation methods.

Method	Space	Win.	Del.	No spk.
CMS [1]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$	$\times$	$\times$	$\checkmark$
Ct.Sketch [12]	$O(\frac{1}{\varepsilon^2} \log \frac{1}{\delta})$	$\times$	$\checkmark$	$\checkmark$
Hokusai [4]	$O(\frac{1}{\varepsilon} \log N \log \frac{1}{\delta})$	$\checkmark$	$\times$	$\checkmark$
ECM [3]	$O(\frac{1}{\varepsilon^2} \log N \log \frac{1}{\delta})$	$\checkmark$	$\times$	$\checkmark$
Sim.Reset [6]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$	approx	$\times$	$\times$
Prop.Dec. [6]	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$	approx	$\times$	partial
SD-CMS	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$	$\checkmark$	$\checkmark$	$\checkmark$
DLS-CMS	$O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$	$\checkmark$	$\checkmark$	$\checkmark$

Just one approach hits every mark - DLS-CMS makes it happen at the same time, regular CMS near-limit expense.

vs. CMS. CMS weighing in at 5.31 KB, lacks support for windowing - also it won't let you remove items help, yet mistakes climb nonstop as the flow stretches longer M . Even when M changes, DLS-CMS keeps its limit fixed at εF .

vs. Count Sketch. supports deletions at 585.94 KB. $500\times$ worse because the spread grows as $1/\varepsilon^2$. DLS-CMS retains deletion support without widening the sketch.

vs. Hokusai. it handles windowed queries using just 21.25 KB. It doesn't remove data, breaks down when older sections are squeezed. That's DLS-CM. Stands $18\times$ more compact, thanks to built-in delete handling.

vs. SD-CMS. That one came before, yet feels packed starting off, the eager-decay model uses 106.21 KB of space along with taking 5.10 s to run.

On another note, DLS-CMS reaches $90.8\times$ space reduction and $109\times$ speedup via lazy-sparse design.

6 Conclusions

DLS-CMS (Decayed Lazy Sparse Count Min Sketch) - compact by design - tracks how often items appear within a sliding time frame. Streams of data vanish completely when needed. This system grows from Decay Augmented Count Min Sketch (DLS CMS) swaps out the one heavy counter grid using two light hash tables - one for Insertions go here, while deletions get their own spot - counters see decay applied without urgency which get opened for real.

The main contributions and findings are as follows.

Space efficiency. When $\varepsilon = \delta = 0.01$, DLS-CMS operates with at just 1.17 KB - smaller than any of the six others tested - it hits $90.8\times$ less space smaller compared to the earlier version SD-CMS, also way below Count Sketch by a factor of $500\times$. Across every value of ε and δ , extra room stays useful - simply put, more space helps. While numbers shift, the benefit doesn't fade; instead, it sticks around. Even when conditions change, having space makes a difference. It works at the edges just like in the middle. Whenever you look in the range, the edge remains tested.

Speed. Lazy decay lowers per-event cost from $O(dw)$ (SD-CMS) to $O(d)$, yielding a $109\times$ speedup with no loss in accuracy. DLS-CMS is competitive in throughput with standard CMS.

Theoretical guarantees. With DLS-CMS, you get limits on how far off results can be it holds that the chance of $\Pr[\hat{f}(x, t) - f_\lambda(x, t) > \varepsilon F] \leq \delta$. Here, the formula $F = 1/(1 - \lambda)$ gives the actual weight of the window. It works differently than Simple Reset does with Proportional Decay, the assurance stays true no matter the input shape - even when tails stretch far. Heavy or not, it makes no difference.

Deletion support. The dual-sketch architecture enables first-class deletion support without expanding the space or relaxing the accuracy, a property not available in any other method at standard CMS asymptotic cost.

Future work. We believe three directions are promising: (a) evaluate DLS-CMS on real network traffic traces (e.g., CAIDA PCAP datasets), to verify the space savings for power-law item distributions; (b) extend the pruning lemma to handle bursty streams, where many items arrive at the same time; and (c) implement DLS-CMS in P4 on a programmable switch, to measure the hardware-level throughput.

References

- [1] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, 55(1):58–75, 2005.
- [2] M. Datar, A. Gionis, P. Indyk, and R. Motwani, "Maintaining stream statistics over sliding windows," *SIAM Journal on Computing*, 31(6):1794–1813, 2002.
- [3] O. Papapetrou, M. Garofalakis, and A. Deligiannakis, "Sketch-based querying of distributed sliding-window data streams," *Proc. VLDB Endowment*, 5(10), 2012.
- [4] S. Matushevych, A. J. Smola, and A. Ahmed, "Hokusai: sketching streams in real time," in *Proc. UAI*, pp. 594–603, 2012.
- [5] A. Shrivastava, A. C. Konig, and M. Bilenko, "Time adaptive sketches (Ada-Sketches) for summarizing data streams," in *Proc. ACM SIGMOD*, pp. 1417–1432, 2016.
- [6] N. Rivetti, Y. Busnel, and A. Mostéfaoui, "Efficiently summarizing data streams over sliding windows," in *IEEE NCA*, 2015.
- [7] Z. Zeng et al., "A survey on sliding window sketch for network measurement," *Computer Networks*, 226:109696, 2023.
- [8] A. Arasu and G. S. Manku, "Approximate counts and quantiles over sliding windows," in *Proc. ACM PODS*, pp. 286–296, 2004.
- [9] R. Yuan, Y. Qiao, et al., "UCL-sketch: learning-augmented frequency estimation over data streams," *arXiv:2412.03611*, 2024.
- [10] S. Sun, J. Zheng, and D. Li, "HEE-Sketch: an efficient sketch for sliding-window frequency estimation over skewed data streams," in *IEEE ISPA/BDCLOUD*, 2019.
- [11] X. Chen et al., "Fine-grained queue measurement in the data plane," in *Proc. ACM CoNEXT*, pp. 15–29, 2019.
- [12] C. Estan and G. Varghese, "New directions in traffic measurement and accounting," in *Proc. ACM SIGCOMM*, pp. 323–336, 2002.
- [13] C. C. Aggarwal, "On biased reservoir sampling in the presence of stream evolution," in *VLDB*, pp. 607–618, 2006.

- [14] K. Wang, L. Zhang, H. Wen, and L. Xu, "A sliding-window DFT based algorithm for parameter estimation of multi-frequency signal," *Signal Processing*, 2019.
- [15] N. Yi, W. Li, Y. Li, B. Sun, and W. Gui, "Estimation of feeding composition content using variable sliding window method," *ISA Transactions*, 2021.
- [16] J. Hou et al., "Dynamic weight adjustment of fusion positioning based on sliding-window geometric data," *Measurement*, 2022.
- [17] V. Jarlow, C. Stylianopoulos, and M. Papatriantafyllou, "QPOPSS: Query and parallelism optimized space-saving for frequent stream elements," in *Proc. IEEE*, 2020.
- [18] J. Wang, J. Lu, and Z. Su, "SWDAKT: Knowledge tracing using sliding window-based dynamic ability perception," *IEEE Access*, 2023.
- [19] Y. Wang, Y. Wang, and C. Chen, "DPSW-Sketch: A differentially private sketch framework for sliding windows," *Technical Report*, 2023.
- [20] S. Nagawanshi et al., "Learning-augmented moment estimation on time-decay models," 2026.
- [21] Y. Wu et al., "SHE: A generic framework for data stream mining over sliding windows," *ACM TKDD*, 2024.